

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

Name: T Gnaneswari

Reg No: 192372183

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

IT Gnaneswari(192372183) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

T Gnaneswari

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTION 1 – SERVLET-BASED STUDENT REGISTRATION SYSTEM

1. Introduction

A college wants to develop an online student registration system where students enter their personal and academic information through a web form. The submitted information is sent to a Java Servlet running on a web server such as Apache Tomcat.

A Java Servlet is a server-side Java program used to receive HTTP requests, process user input, interact with databases or other services, and send an HTTP response back to the browser.

2. Architecture of a Servlet-Based Application

The basic servlet architecture is:

Student / Browser → HTML Form → Web Server / Servlet Container → Servlet → Database / Business Logic → HTTP Response → Browser

The major components are:

Client / Browser:

The student opens the registration page and enters information such as name, registration number, department, and email.

HTML Form:

The form collects the student information and sends it to the servlet using either the GET or POST HTTP method.

Web Server / Servlet Container:

Apache Tomcat receives the HTTP request. The servlet container identifies the appropriate servlet and forwards the request to it.

Servlet:

The servlet receives the form values through the `HttpServletRequest` object and processes the data.

Database:

The servlet can optionally store the submitted student information in a database using JDBC.

HTTP Response:

After processing the request, the servlet generates a response using `HttpServletResponse`.

3. Student Registration HTML Form

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Student Registration</title>
</head>
<body>
<h2>College Student Registration</h2>
<form action="StudentServlet" method="post">
    Name:
    <input type="text" name="name" required>
    <br><br>

    Registration Number:
    <input type="text" name="regno" required>
    <br><br>

    Department:
    <input type="text" name="department" required>
    <br><br>

    Email:
    <input type="email" name="email" required>
    <br><br>

    Password:
    <input type="password" name="password" required>
    <br><br>

    <input type="submit" value="Register">

</form>

</body>
</html>
```

4. Servlet Program Using GET and POST

```
import java.io.*;
```

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/StudentServlet")
public class StudentServlet extends HttpServlet {

    // Called only once when servlet is created
    @Override
    public void init() throws ServletException {
        System.out.println("Student Servlet Initialized");
    }

    // Handles GET request
    @Override
    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String regno = request.getParameter("regno");

        out.println("<html>");
        out.println("<body>");
        out.println("<h2>GET Request</h2>");
        out.println("<p>Name: " + name + "</p>");
        out.println("<p>Registration Number: " + regno + "</p>");
        out.println("</body>");
        out.println("</html>");
    }
}
```

```

// Handles POST request
@Override
protected void doPost(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    String name = request.getParameter("name");
    String regno = request.getParameter("regno");
    String department = request.getParameter("department");
    String email = request.getParameter("email");
    String password = request.getParameter("password");

    out.println("<html>");
    out.println("<body>");

    out.println("<h2>Student Registration Successful</h2>");

    out.println("<p>Name: " + name + "</p>");
    out.println("<p>Registration Number: " + regno + "</p>");
    out.println("<p>Department: " + department + "</p>");
    out.println("<p>Email: " + email + "</p>");

    out.println("</body>");
    out.println("</html>");
}

// Called once before servlet is removed
@Override
public void destroy() {

```

```
        System.out.println("Student Servlet Destroyed");
    }
}
```

5. GET Method

The GET method is normally used to retrieve information from the server.

Example:

StudentServlet?name=Gnaneswari®no=12345

The submitted values are appended to the URL as query parameters.

Example Java method:

```
protected void doGet(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException {

    String name = request.getParameter("name");

    response.getWriter().println("Student Name: " + name);
}
```

Advantages of GET

GET is simple and suitable for retrieving information.

GET URLs can be bookmarked.

GET requests can be cached by browsers.

Disadvantages of GET

The submitted data is visible in the URL.

The URL length is limited.

Sensitive data such as passwords should never be submitted through GET.

6. POST Method

POST sends the data inside the body of the HTTP request rather than placing it in the URL.

Example:

```
protected void doPost(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException {

    String name = request.getParameter("name");
```

```
String email = request.getParameter("email");
```

```
    response.getWriter().println("Registration Successful");  
}
```

POST is normally used for registration forms, login forms, file uploads, payment details, and database updates.

7. Servlet Life Cycle

A servlet has three important life-cycle stages:

init() → service() → destroy()

init()

The init() method is called once when the servlet is first created by the servlet container.

```
public void init() throws ServletException {  
    System.out.println("Servlet Initialized");  
}
```

It can be used for initialization activities such as loading configuration values or establishing resources.

service()

The service() method handles client requests.

For HttpServlet, the servlet container automatically calls methods such as:

```
doGet()  
doPost()  
doPut()  
doDelete()
```

For example:

GET request → doGet()

POST request → doPost()

The service() method may execute many times while the servlet remains active.

destroy()

The destroy() method is executed once before the servlet is removed from memory.

```
public void destroy() {  
    System.out.println("Servlet Destroyed");  
}
```

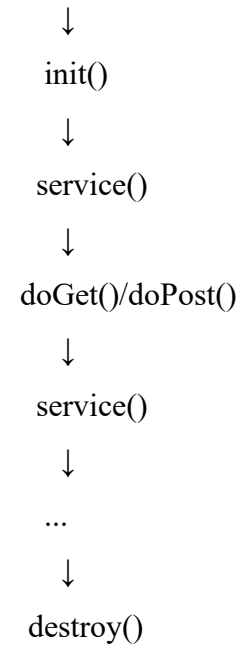


```
}
```

It is generally used to release resources such as database connections, files, or other allocated resources.

8. Servlet Life Cycle Flow

Servlet Loaded



9. GET vs POST for Sensitive Data

For registration information, login credentials, and passwords, **POST is more appropriate than GET**.

With GET:

```
/login?username=user&password=12345
```

Sensitive information can appear in browser history, server logs, copied URLs, and other locations.

With POST, parameters are placed inside the HTTP request body and are not displayed in the browser address bar.

However, POST alone does **not encrypt the information**. HTTPS must also be used so that sensitive data is protected during network transmission.

Therefore:

POST + HTTPS is the recommended approach for sensitive form data.

Conclusion

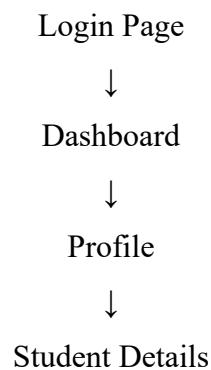
Servlets provide an effective server-side mechanism for processing web requests in Java applications. In the student registration system, the browser submits student information to a servlet, which processes it and generates an HTTP response. GET is suitable for retrieving non-sensitive information, whereas POST is preferred for registration and other sensitive operations.

QUESTION 2 – SESSION MANAGEMENT USING HTTPSESSION AND COOKIES

1. Introduction

HTTP is a stateless protocol. This means that each request sent by a user is treated independently.

For example:



Without session management, the web server would not automatically know that all these requests are coming from the same authenticated user.

Therefore, session tracking is used to maintain the user's identity across multiple pages.

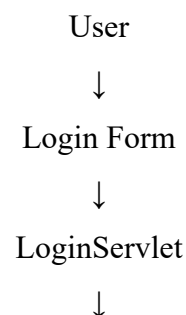
Two common techniques are:

HttpSession

and

Cookies

2. Session Management Architecture



Validate Username and Password



Create HttpSession



Generate Session ID



Session ID stored in Cookie



Dashboard / Other Pages

3. Login HTML Page

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>User Login</title>
```

```
</head>
```

```
<body>
```

```
<h2>Login</h2>
```

```
<form action="LoginServlet" method="post">
```

Username:

```
<input type="text" name="username" required>
```

```
<br><br>
```

Password:

```
<input type="password" name="password" required>
```

```
<br><br>
```

```
<input type="submit" value="Login">
```

```
</form>
```

```
</body>
```

```
</html>
```

4. Login Servlet Using HttpSession

```
import java.io.*;
```

```
import jakarta.servlet.*;
```

```
import jakarta.servlet.http.*;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
@WebServlet("/LoginServlet")
```

```
public class LoginServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {
```

```
        String username = request.getParameter("username");
```

```
        String password = request.getParameter("password");
```

```
        if(username.equals("admin") &&
           password.equals("1234")) {
```

```
            HttpSession session = request.getSession();
```

```
            session.setAttribute("username", username);
```

```
            session.setMaxInactiveInterval(30 * 60);
```

```
            response.sendRedirect("DashboardServlet");
```

```
        } else {
```

```
            response.getWriter().println(
                "<h3>Invalid Username or Password</h3>")
```

```

        );
    }
}
}

```

Here:

```
request.getSession();
```

creates a new session if one does not already exist.

```
session.setAttribute("username", username);
```

stores the username inside the server-side session.

5. Dashboard Servlet

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/DashboardServlet")
public class DashboardServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session =
            request.getSession(false);

        if(session == null ||
           session.getAttribute("username") == null) {

            response.sendRedirect("login.html");

            return;
        }

        String username =

```

```

        (String) session.getAttribute("username");

    PrintWriter out = response.getWriter();

    out.println("<html>");
    out.println("<body>");

    out.println("<h2>Welcome " +
        username + "</h2>");

    out.println(
        "<a href='LogoutServlet'>Logout</a>"
    );

    out.println("</body>");
    out.println("</html>");
}
}

```

The statement:

```
request.getSession(false);
```

returns the existing session without creating a new session.

6. Logout Servlet

```

import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/LogoutServlet")
public class LogoutServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

```

```

HttpSession session =
    request.getSession(false);

if(session != null) {
    session.invalidate();
}

response.sendRedirect("login.html");
}
}

```

The following statement destroys the session:

```
session.invalidate();
```

After logout, the user must log in again.

7. Cookie Implementation

Cookies are small pieces of information stored in the user's browser.

A servlet can create a cookie as follows:

```

Cookie userCookie =
    new Cookie("username", "Ganeswari");

```

```
userCookie.setMaxAge(60 * 60);
```

```
response.addCookie(userCookie);
```

The browser stores the cookie.

The servlet can retrieve cookies using:

```
Cookie[] cookies = request.getCookies();
```

```
if(cookies != null) {
```

```
    for(Cookie cookie : cookies) {
```

```
        if(cookie.getName().equals("username")) {
```

```
            String username =
                cookie.getValue();

```

```

        response.getWriter()
            .println("Welcome " + username);
    }
}
}

```

8. How Session Tracking Works

When the user successfully logs in, the servlet container creates a unique session.

For example:

Session ID = AB123XYZ987

The session data may be stored on the server.

AB123XYZ987

Username = Gnaneswari

Role = Student

Authenticated = true

The browser usually receives a cookie such as:

JSESSIONID=AB123XYZ987

During subsequent requests, the browser automatically sends the JSESSIONID cookie.

The server uses this ID to locate the corresponding session.

Therefore:

Browser

↓

JSESSIONID

↓

Server

↓

Identify Existing Session

↓

Allow User to Access Protected Pages

9. HttpSession vs Cookies

Feature	HttpSession	Cookies
Storage	Server side	Client/browser side
Security	More secure for application data	Less suitable for sensitive data

Feature	HttpSession	Cookies
Capacity	Can store larger objects	Usually small
Lifetime	Until timeout/logout	Can remain until expiry
User modification	Difficult	Users may modify cookies
Common use	Authentication/session state	Preferences/session ID
Data type	Java objects	String values
Network transfer	Only ID normally transferred	Cookie data sent with requests

HttpSession is generally preferred for authentication because sensitive information can remain on the server.

Cookies are commonly used to store the session identifier or non-sensitive preferences.

A secure application should configure authentication cookies using protections such as HttpOnly, Secure, and appropriate SameSite settings.

10. Conclusion

Session management is necessary because HTTP itself is stateless. HttpSession allows the server to maintain authenticated user information across multiple pages. Cookies are stored in the browser and are commonly used for session identifiers or preferences. For authentication, server-side sessions are generally safer because sensitive application state does not need to be stored directly in the browser.

QUESTION 3 – JDBC-BASED E-COMMERCE PRODUCT MANAGEMENT

1. Introduction

An e-commerce application must store product information such as product ID, product name, price, category, and quantity.

Java Database Connectivity, commonly called **JDBC**, allows Java programs to connect to relational databases such as:

MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.

JDBC can perform operations including:

INSERT

SELECT

UPDATE

DELETE

2. JDBC Architecture

Basic JDBC architecture:

Java Application



JDBC API



JDBC Driver



Database Management System



Database

Java Application

Contains the application or servlet code.

JDBC API

Provides standard Java interfaces and classes for database communication.

Important JDBC classes/interfaces include:

DriverManager

Connection

Statement

PreparedStatement

ResultSet

SQLException

JDBC Driver

The JDBC driver translates Java JDBC calls into database-specific communication.

Database

Stores the application information.

3. Product Database Table

Example MySQL database:

CREATE DATABASE ecommerce;

Select the database:

USE ecommerce;

Create the product table:

CREATE TABLE products (

```
id INT PRIMARY KEY AUTO_INCREMENT,  
name VARCHAR(100),  
price DECIMAL(10,2),  
category VARCHAR(50),  
quantity INT  
);
```

4. JDBC Database Connection

Example connection:

```
import java.sql.*;
```

```
public class DatabaseConnection {  
  
    public static Connection getConnection()  
        throws SQLException {  
  
        String url =  
            "jdbc:mysql://localhost:3306/ecommerce";  
  
        String username = "root";  
  
        String password = "root";  
  
        return DriverManager.getConnection(  
            url,  
            username,  
            password  
        );  
    }  
}
```

Modern JDBC drivers are generally discovered automatically when their driver JAR is on the application's classpath.

In older JDBC applications, the driver may also be explicitly loaded using:

```
Class.forName(  
    "com.mysql.cj.jdbc.Driver"
```

```
);
```

5. Program to Insert Product Data

```
import java.sql.*;
```

```
public class InsertProduct {
```

```
    public static void main(String[] args) {
```

```
        String url =
```

```
            "jdbc:mysql://localhost:3306/ecommerce";
```

```
        String username = "root";
```

```
        String password = "root";
```

```
        String sql =
```

```
            "INSERT INTO products " +
```

```
            "(name, price, category, quantity) " +
```

```
            "VALUES (?, ?, ?, ?)";
```

```
        try (
```

```
            Connection con =
```

```
                DriverManager.getConnection(
```

```
                    url,
```

```
                    username,
```

```
                    password
```

```
                );
```

```
            PreparedStatement ps =
```

```
                con.prepareStatement(sql)
```

```
        ) {
```

```
            ps.setString(1, "Laptop");
```

```
            ps.setDouble(2, 55000.00);
```

```
            ps.setString(3, "Electronics");
```

```

        ps.setInt(4, 10);

        int rows = ps.executeUpdate();

        if(rows > 0) {
            System.out.println(
                "Product inserted successfully."
            );
        }

    } catch(SQLException e) {

        System.out.println(
            "Database Error: " +
            e.getMessage()
        );

        e.printStackTrace();
    }
}
}

```

6. Why PreparedStatement Is Used

PreparedStatement is recommended instead of constructing SQL using direct string concatenation.

For example:

PreparedStatement ps =

```

    con.prepareStatement(
        "INSERT INTO products " +
        "(name, price) VALUES (?, ?)"
    );

```

The ? characters represent parameter placeholders.

Values are assigned using:

```

ps.setString(1, "Laptop");
ps.setDouble(2, 55000);

```

Prepared statements make SQL code easier to manage and significantly reduce the risk of SQL injection when input values are parameterized correctly.

7. Program to Retrieve Product Data

```
import java.sql.*;

public class ViewProducts {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/ecommerce";

        String username = "root";
        String password = "root";

        String sql =
            "SELECT id, name, price, " +
            "category, quantity FROM products";

        try (
            Connection con =
                DriverManager.getConnection(
                    url,
                    username,
                    password
                );

            PreparedStatement ps =
                con.prepareStatement(sql);

            ResultSet rs =
                ps.executeQuery()
        ) {
```

```
System.out.println(
    "ID\tName\tPrice\tCategory\tQuantity"
);

while(rs.next()) {

    int id =
        rs.getInt("id");

    String name =
        rs.getString("name");

    double price =
        rs.getDouble("price");

    String category =
        rs.getString("category");

    int quantity =
        rs.getInt("quantity");

    System.out.println(
        id + "\t" +
        name + "\t" +
        price + "\t" +
        category + "\t" +
        quantity
    );
}

} catch(SQLException e) {

    System.out.println(
        "Unable to retrieve products."
```

```

        );

        e.printStackTrace();
    }
}

```

8. JDBC Connectivity Steps

The database connectivity process can be represented as:

Java Program



Import JDBC Packages



Load / Register JDBC Driver



Establish Connection



Create PreparedStatement



Execute SQL Query



Process ResultSet



Close Resources

Step 1 – Import JDBC Package

```
import java.sql.*;
```

Step 2 – Load JDBC Driver

Older applications may use:

```

Class.forName(
    "com.mysql.cj.jdbc.Driver"
);

```

With modern JDBC drivers, automatic driver loading is normally supported.

Step 3 – Establish Connection

Connection con =

```
DriverManager.getConnection(
```



```
url,  
username,  
password  
);
```

Step 4 – Prepare SQL Statement

```
PreparedStatement ps =  
    con.prepareStatement(  
        "SELECT * FROM products"  
    );
```

Step 5 – Execute SQL

For INSERT, UPDATE, and DELETE:

```
int rows =  
    ps.executeUpdate();
```

For SELECT:

```
ResultSet rs =  
    ps.executeQuery();
```

Step 6 – Process the Result

```
while(rs.next()) {  
  
    System.out.println(  
        rs.getString("name")  
    );  
}
```

Step 7 – Close Database Resources

Using try-with-resources is recommended:

```
try (  
    Connection con = ...;  
    PreparedStatement ps = ...;  
    ResultSet rs = ...  
) {  
  
    // Database operations  
  
}
```

Java automatically closes the resources when execution leaves the try block.

9. Exception Handling in JDBC

Database operations may fail for several reasons, including:

Incorrect database URL

Incorrect username/password

Database server unavailable

Invalid SQL syntax

Constraint violation

Network problem

Missing JDBC driver

JDBC operations should therefore be placed inside proper exception handling.

Example:

```
try {
```

```
    Connection con =
```

```
        DriverManager.getConnection(
```

```
            url,
```

```
            username,
```

```
            password
```

```
        );
```

```
} catch(SQLException e) {
```

```
    System.out.println(
```

```
        "Database connection failed."
```

```
    );
```

```
    System.out.println(
```

```
        e.getMessage()
```

```
    );
```

```
}
```

Detailed development diagnostics may use:

```
e.printStackTrace();
```



```

        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        ps.setString(1, name);
        ps.setDouble(2, price);
        ps.setString(3, category);
        ps.setInt(4, quantity);

        ps.executeUpdate();

        System.out.println(
            "Product inserted successfully."
        );

    } catch(SQLException e) {

        System.out.println(
            "Insert failed: " +
            e.getMessage()
        );
    }
}

public static void displayProducts() {

    String sql =
        "SELECT * FROM products";

    try (
        Connection con =
            DriverManager.getConnection(
                URL, USER, PASSWORD
            );

```

```
PreparedStatement ps =
    con.prepareStatement(sql);

ResultSet rs =
    ps.executeQuery()
) {

    while(rs.next()) {

        System.out.println(
            "Product ID: " +
            rs.getInt("id")
        );

        System.out.println(
            "Name: " +
            rs.getString("name")
        );

        System.out.println(
            "Price: " +
            rs.getDouble("price")
        );

        System.out.println(
            "Category: " +
            rs.getString("category")
        );

        System.out.println(
            "Quantity: " +
            rs.getInt("quantity")
        );
    }
}
```

```

        System.out.println(
            "-----"
        );
    }

} catch(SQLException e) {

    System.out.println(
        "Retrieve failed: " +
        e.getMessage()
    );
}

}

public static void main(String[] args) {

    insertProduct(
        "Laptop",
        55000,
        "Electronics",
        10
    );

    insertProduct(
        "Keyboard",
        1500,
        "Accessories",
        20
    );

    displayProducts();
}
}

```

11. Expected Output

Product inserted successfully.

Product inserted successfully.

Product ID: 1

Name: Laptop

Price: 55000.0

Category: Electronics

Quantity: 10

Product ID: 2

Name: Keyboard

Price: 1500.0

Category: Accessories

Quantity: 20

12. Important JDBC Concepts Used

Component	Purpose
DriverManager	Establishes database connection
Connection	Represents database connection
PreparedStatement	Executes parameterized SQL
ResultSet	Stores SELECT query results
executeUpdate()	Executes INSERT, UPDATE and DELETE
executeQuery()	Executes SELECT statements
SQLException	Handles database-related errors

13. Final Conclusion

JDBC provides a standardized interface through which Java applications can communicate with relational databases. In the e-commerce system, JDBC can be used to insert, retrieve, modify, and delete product records.